i) Subarrays with Bitwise OR 1

ii) Valid Sudoko

iii) Gray code 1

iv) power function impl

v) discussion on bee

Q1. Subarrays with Bitwise OR 1

Given an array B of length A with element 1 or 0, find the no. of subarrays with OR = 1.

ex: $A = 3$

$B = [1\ 0\ 1]$

B = [1] [0] [1] [1 0] [0 1] [1 0 1]

1 1 1 1 1

O/P = 5

Bruteforce

1) try for all possible subarrays

(i, j)

start → i, end → j

for(i →)
    for(j →)
        for(k ⇒ i to j)

OR

TC ⇒ $O(N^3)$

Optimised

0 0 0 0 0 0 0 1 ⇒ 1

0 0 0 0 1 0 0 1 ⇒ 1

1 1 1 1 1 1 1 1 ⇒ 1

0 0 0 0 0 0 0 0 ⇒ 0

* if subarray contains even single '1' bit then OR for entire subarray = 1

⇒ no. of subarrays whose OR is 1 = no. of subarrays containing atleast single '1'.

$$\text{arr} \Rightarrow \begin{array}{ccccc} 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 2 & 3 & 4 \end{array}$$

subarray = 3 + 5

$$\text{arr} \Rightarrow \begin{array}{ccccc} 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 2 & 3 & 4 \end{array}$$

last = (i+1) = 3.

total = 3 + 3 + 3
= 9

arr ⇒ $\begin{array}{ccccccc} 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 2 & 3 & 4 & 5 & 6 \end{array}$

[i+1]
last = 6

total = 3 + 3 + 3 + 6 + 6

pseudo

last = 0

last = (i+1) if (A[i] == 1)

total = total + last

$$\boxed{\begin{array}{ll} TC \Rightarrow & O(N) \\ SC \Rightarrow & O(1) \end{array}}$$

Q2 Gray code :-

Gray code is a binary numeral system where two successive values differ in only one bit. Gray code sequences starts from 0

$A = 2 \Rightarrow$

0 0 – 0
0 1 – 1
1 1 – 3
1 0 – 2

$A = 3 \Rightarrow$

0 0 0 – 0
0 0 1 – 1
0 1 1 – 3
0 1 0 – 2
1 1 0 – 6
1 1 1 – 7
1 0 1 – 5
1 0 0 – 4

$g(2)$

$r[g(2)]$

$A = 4 \Rightarrow$

0 0 0 0 – 0
0 0 0 1 – 1
0 0 1 1 – 3
0 0 1 0 – 2
0 1 1 0 – 6
0 1 1 1 – 7
0 1 0 1 – 5
0 1 0 0 – 4
1 1 0 0 – 12

$g(3)$

1	1	0	1
1	1	1	1
1	1	1	0
1	0	1	0
1	0	1	1
1	0	0	1
1	0	0	0

→ r[g(3)]

main logic

grey(N) => '0' + grey(N-1)
& '1' + rever(grey(N-1))

base case → grey(1) : 0
1

gery(2) : 0 0
0 1
1 1
1 0

grey(3) =>

0	0	0
0	0	1
0	1	1
0	1	0
1	1	0
1	1	1
1	0	1
1	0	0

[0, 1] → [00, 01, 11, 10]

[000, 001, 011, 010,
110, 111, 101, 100]

Q3. Valid Sodoku

j=	0	1	2	3	4	5	6	7	8
i= 0	5	3			7				
1	6			1	9	5			
2		9	8					6	
	8				6				3
	4			8		3			1
	7				2				6
		6					2	8	
				4	1	9			5
					8			7	9

duplicates

i) 9 x 9 matrix ✓

~~ii) 9 blocks (3x3) matrix~~

iii) any row [1-9] no duplicates

iv) any colm [1-9] no duplicates

v) any matrix (3x3) contains [1-9] no duplicates.

i) for (i = 0 → 8) {

N^2 ← for (j = 0 → 8) {

check for duplicats

}

}

row

ii) for ( ) {

for ( ) {

N^2 ←

}

column

iii) for (i → 0) {

N^2 ← for (j → 0) {

}

}

(3×3)

9

TC → $3(N^2)$ → 3×81 → $\boxed{243}$ $O(1)$

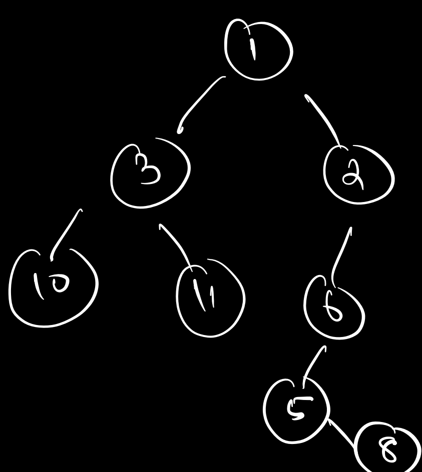
SC => O(1)

$\boxed{TC \ \& \ SC \rightarrow O(1)}$

=> Tree Discussion :

pre order	Inorder	postOrder
=> Root	left	left
left	root	right
right	right	root
[N L R]	[L N R]	[L R N]

1)



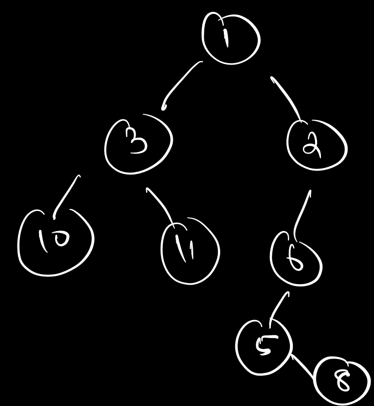
Pre => 1 3 10 11 2 6 5 8

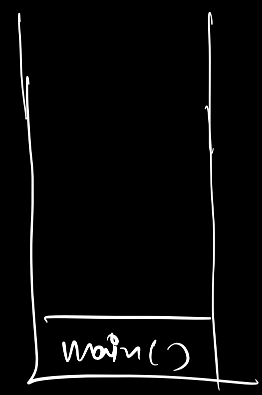
In => 10 3 11 1 5 8 6 2

$\underbrace{10 \ 3 \ 11}_{LST} \ 1 \ \underbrace{5 \ 8 \ 6 \ 2}_{RST}$

Post => 10 11 3 8 5 6 2 1

```
void preOrder(root) {
    if(root == null)
        return

    print(root.data)

    preorder(root.left)

    preorder(root.right)

}
```



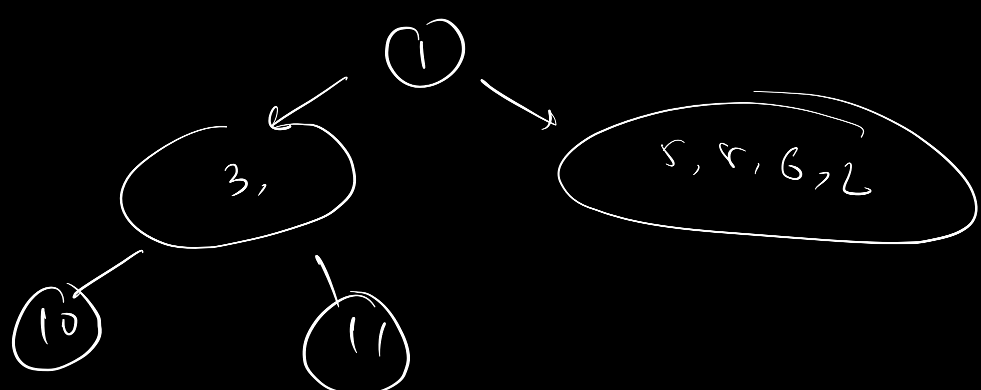


1 3 10 11 2 6 5 8

Pre ⇒ 1 3 10 11 2 6 5 8

In ⇒ 10 3 11 1 5 8 6 2 RST

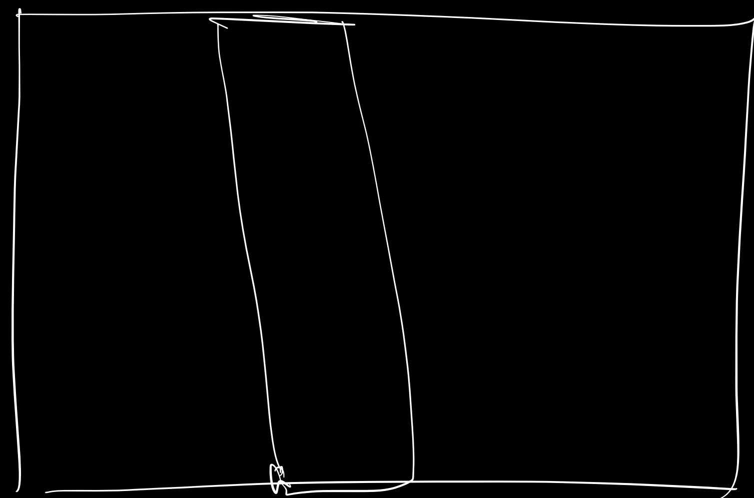
Post ⇒ 10 11 3 8 5 6 2 1



Order history

Table

[Read heavy table]



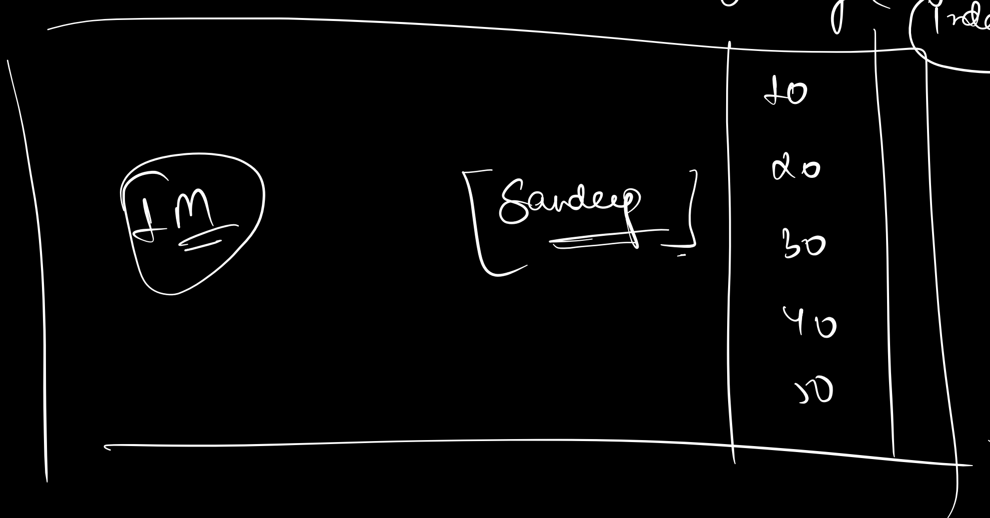
session table

5k

10k

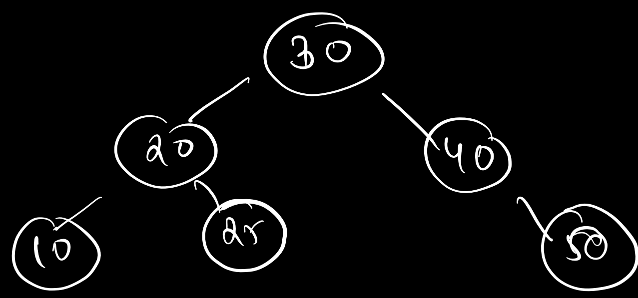
orderby

Indexing



log N

search



log N

costly

20 → 1M

23/24 → 10M

27 → 100M

30 → 1B

34 → 10B

37 → 100B

search → O(N)
insert → O(1)

read → O(N)
write → O(1)

$\boxed{\begin{array}{l}\text{read} \to O(\log N) \\ \text{write} \to O(\log N)\end{array}}$

logging

logged

search